How a Developer Runs This Application

Acme Retail Internal Developer Platform | Inventory Management System (IMS) reference app

Summary. Two kinds of developer use this repository. An **app developer** works on the IMS service itself. A **new-team developer** uses the platform to create their own service with CI/CD, security scanning and infrastructure already wired in. Both flows are covered below, followed by what happens on every push and how to try the infrastructure without a cloud account.

1. Prerequisites

Tool	Needed for	Required?
Git	Clone and push	Yes
Python 3.12	Run and test the app	Yes
Docker	Build the container; run LocalStack	Yes (for container / infra steps)
Terraform 1.6+	Infrastructure checks	Optional
Trivy, Checkov, Gitleaks	Local security scans	Optional (skipped if missing)
AWS CLI	Inspect LocalStack resources	Optional

2. Developer working on the IMS app

2.1 One-time setup

```
git clone https://github.com/SagarMani56/Test.git
cd Test/app
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
```

2.2 Run it locally

```
uvicorn src.main:app --reload
```

- Interactive API page: `http://localhost:8000/docs`
- Health check: `http://localhost:8000/health` returns `{"status":"ok"}`

2.3 Try the API

```
curl -X POST localhost:8000/items -H "Content-Type: application/json" \
  -d '{"sku":"A1","name":"Mug","quantity":5}'
curl localhost:8000/items/A1
curl -X PUT "localhost:8000/items/A1/stock?delta=-2"
```

> **Note:** items are kept in memory, so data is lost on restart. The DynamoDB table created by the Terraform module is provisioned but not yet wired into the application code.

2.4 Run as a container (the image CI builds)

```
docker build -t acme-ims .
docker run -p 8000:8000 acme-ims
```

2.5 Check your work before pushing

```
ruff check . && pytest --cov=src --cov-fail-under=80

# From the repository root - runs every check (skips tools you have not installed):
make validate
```

3. Developer on a new team (the platform's purpose)

```
make new-service NAME=orders-service TEAM=commerce
cd orders-service/app
pip install -r requirements-dev.txt
pytest
uvicorn src.main:app --reload
```

The scaffolder creates a ready-made repository containing:

Item	What it is
app/	Health-check service, a passing test, a hardened Dockerfile
.github/workflows/ci.yml	About 15 lines that call the platform's reusable workflows
infra/main.tf	One call to the app-platform Terraform module
docs/ai-spec.md	Service spec that inherits the platform specs
CODEOWNERS, .gitignore, .gitleaks.toml	Ownership and secret-scanning defaults

The developer then replaces the example code in `app/src/`, adds tests, pushes to GitHub (CI starts automatically) and fills in `docs/ai-spec.md`. No pipeline code and no raw cloud resources are written by the team.

4. What happens when they push

```
git push / open pull request
  |
  +-- ci.yml calls the platform's reusable workflows:
  |     lint -> tests (80% coverage gate) -> Gitleaks -> build image -> Trivy scans
  |     Terraform fmt/validate -> Checkov -> (optional) LocalStack apply
  |
  +-- any failure blocks the merge
  +-- tag v1.0.0 -> release workflow pushes the image to GHCR with an SBOM
```

Gate	Tool	Blocks merge?
Code style	ruff	Yes
Tests and coverage (80% minimum)	pytest-cov	Yes
Secrets in git history	Gitleaks	Yes
HIGH / CRITICAL vulnerabilities (dependencies and image)	Trivy	Yes
Infrastructure misconfiguration	Checkov	Yes
Terraform format and validity	terraform fmt / validate	Yes

5. Trying the infrastructure without a cloud account

```
make localstack EX=ims            # or EX=team-b-orders
```

This starts LocalStack in Docker, waits until it is ready, applies the Terraform (network, DynamoDB table, S3 bucket), prints the outputs, and destroys everything when you press Enter. To inspect resources while it is paused:

```
aws --endpoint-url http://localhost:4566 dynamodb list-tables
aws --endpoint-url http://localhost:4566 s3 ls
```

Limits to be aware of. The LocalStack apply is still being stabilised in the pipeline, so treat it as under test until it runs green. The container registry and ECS service are only validated and scanned, never deployed, because LocalStack's free edition does not emulate them. Deploying to real AWS means setting `use_localstack=false` and `enable_compute=true`.

6. Command quick reference

I want to...	Command
Run tests	`make test`
Run every local check	`make validate`
Create a new service	`make new-service NAME=<name> TEAM=<team>`
Apply Terraform locally (no cloud)	`make localstack EX=ims`
Start the app with auto-reload	`uvicorn src.main:app --reload` (from app/)
Build the container	`docker build -t acme-ims app`

7. Where to read more

- `README.md` - 60-second tour and problem-to-solution table
- `docs/HOWTO.md` - full step-by-step tutorial, including pushing and submission
- `docs/architecture/architecture.md` - Mermaid diagrams
- `docs/ai-specifications/` - the six AI Engineering Specifications
- `docs/VALIDATION.md` - validation evidence and what to run yourself